

GitHub Actions & Workflow Generation

DevOps & Infrastructure | Intermediate

Agenda

Time	Topic
0:00–0:08	Actions model
0:08–0:18	Generate CI
0:18–0:28	Debug a failed step
0:28–0:38	Reusable and custom actions
0:38–0:50	Bounded pipeline demo
0:50–1:00	Security, <code>gh-aw</code> , lab handoff

Actions model

Part	Meaning
Workflow	YAML automation in <code>.github/workflows/</code>
Event	Trigger such as <code>push</code> or <code>pull_request</code>
Job	Steps on a runner
Step	An action or command
Secret	Sensitive value outside source
Artifact	Files retained from a job

A workflow is code with an execution boundary

Workflow YAML chooses when automation runs, what it can read, and where it can write. Review it like application code, not as deployment plumbing.

Question	Why it matters
What event starts it?	Untrusted pull-request data changes the threat model
Which runner executes it?	The runner sets available tools and isolation
Which token permissions exist?	Broad tokens turn small mistakes into writes
Which outputs leave the job?	Artifacts and logs can expose data

The generated YAML is only the first draft.

Triggers decide who influences execution

`pull_request` runs in the contributor context with restricted permissions.

`pull_request_target` runs in the base-repository context and needs extra care.

Never combine a privileged trigger with untrusted checkout, scripts, or pull-request metadata unless the approved design specifically requires it and the controls are reviewed. The safest workflow often separates untrusted validation from privileged follow-up work.

Build feedback in layers

Start with the fastest trustworthy signal, then add slower checks.

```
graph TD; A[Format and lint] --> B[Focused unit tests]; B --> C[Build and integration checks]; C --> D[Deployment approval];
```

Format and lint
↓
Focused unit tests
↓
Build and integration checks
↓
Deployment approval

Each layer should state what it proves and what it does not prove. A green lint job does not prove the build, and a successful build does not approve a deployment.

Generate from repository facts

Read package.json and existing tests. Create ci.yml for pull requests and main.
Use Node.js 20, npm ci, existing lint and test scripts, npm caching, contents: read,
and a test-results artifact. Do not add dependencies or deployment steps.

Check every assumption against the project.

Debug with useful context

Provide the failed step, error, and workflow file. Ask for the smallest fix and the follow-up check.

This workflow fails at "Run tests." Here is the step output and ci.yml.
Identify the cause, propose the smallest fix, and explain what to verify.

Security review

- Set explicit least-privilege `permissions`.
- Keep values in secrets or use OIDC.
- Review triggers, especially `pull_request_target`.
- Pin approved third-party actions to immutable SHAs in production.
- Check commands, runners, artifacts, concurrency, and deployment gates.

```
permissions:  
  contents: read
```

Agentic Workflows

```
gh extension install github/gh-aw  
gh aw init daily-repo-status  
gh aw compile daily-repo-status  
gh aw logs daily-repo-status
```

Review Markdown source and compiled `.lock.yml`. Keep permissions read-only unless a reviewed `safe-outputs` write is required.

Lab

Build CI, design CD, fix supplied broken workflows, and create a JavaScript action. `gh-aw` is optional and requires approval.